Lecture 5b: Floating Point

Session 6 · Wed Sep 16, 2026

Contents

Integers Cannot Do This . . . 1
Where We Are . . . 2
The Same Idea You Already Use . . . 2
A Float Is Three Fields . . . 3
Turning 5.75 Into Thirty-Two Bits . . . 3
Decoding One Float . . . 4
See the Bits Yourself . . . 4
Pulling the Three Fields Apart . . . 4
Run It: Eight Values, Thirty-Two Bits . . . 5
Why 0.1 Is Not 0.1 . . . 5
Run It: What 0.1 Actually Holds . . . 5
Never Compare Floats with == . . . 5
Floats Are Not Evenly Spaced . . . 6
Run It: The Gap Grows With the Number . . . 6
Where a Float Stops Counting . . . 6
Big Numbers Eat Small Ones . . . 7
Floating Point Is Not Real Arithmetic . . . 7
Run It: Same Numbers, Different Order . . . 7
Rounding . . . 7
Three Ways to Lose the Fraction . . . 8
Special Values . . . 8
Run It: The Ends of the Range . . . 8
AI Systems Connection . . . 9
BF16 Is FP32 With the Tail Cut Off . . . 9
Two Consequences You Will Hear About . . . 9
The Unit in One Slide . . . 10
Practice . . . 10
Reading . . . 10

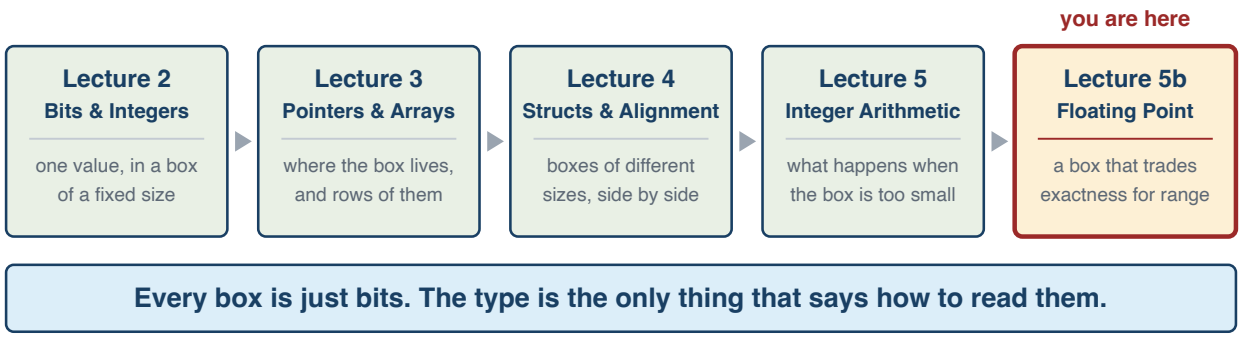
Integers Cannot Do This

Everything so far has been whole numbers in a fixed-size box. Two things that box cannot hold:

```
int half = 3 / 2;      // 1 — the half is not rounded away, there was never room
int atoms = 602214076000000000000000;   // does not fit in 32 bits. Or 64.
```

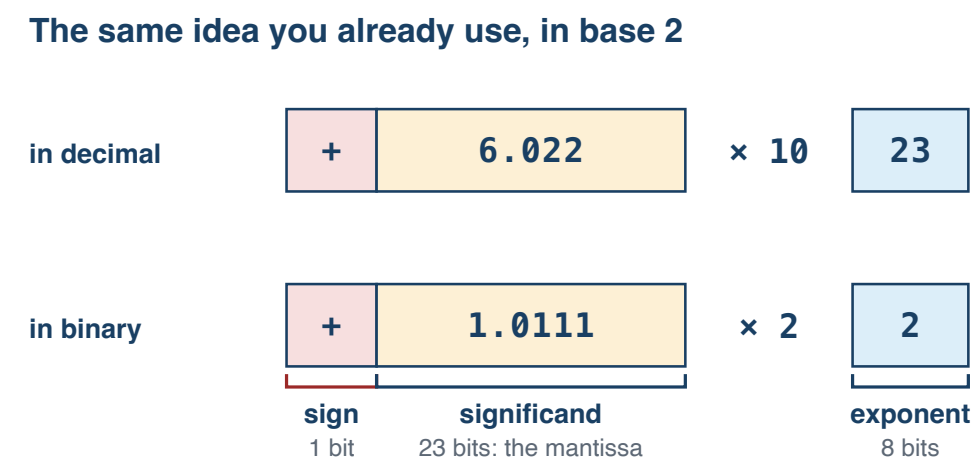
We need fractions, and we need range. Adding more bits to an `int` buys range and no fractions at all. Floating point buys both, and pays for them in exactness.

Where We Are



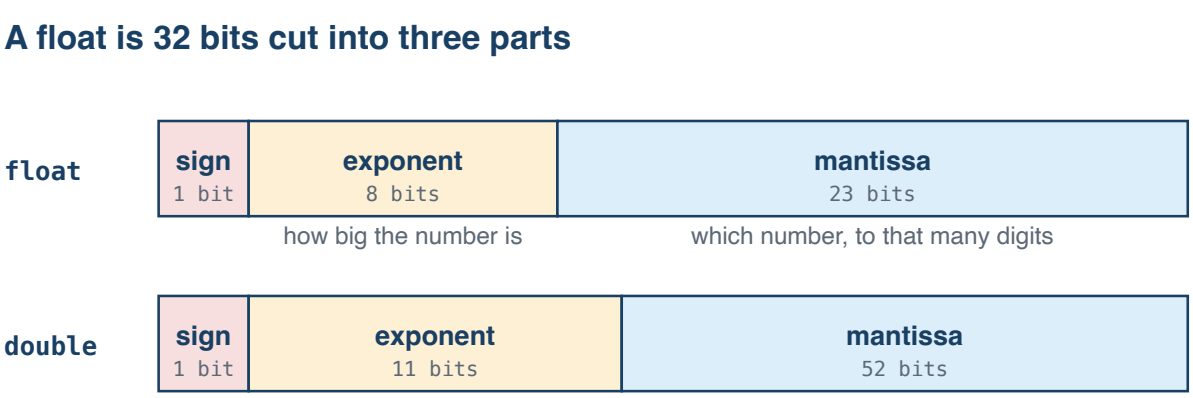
Four lectures of boxes that hold whole numbers exactly. This is the last one, and the only box that gives up exactness — on purpose, to buy range.

The Same Idea You Already Use



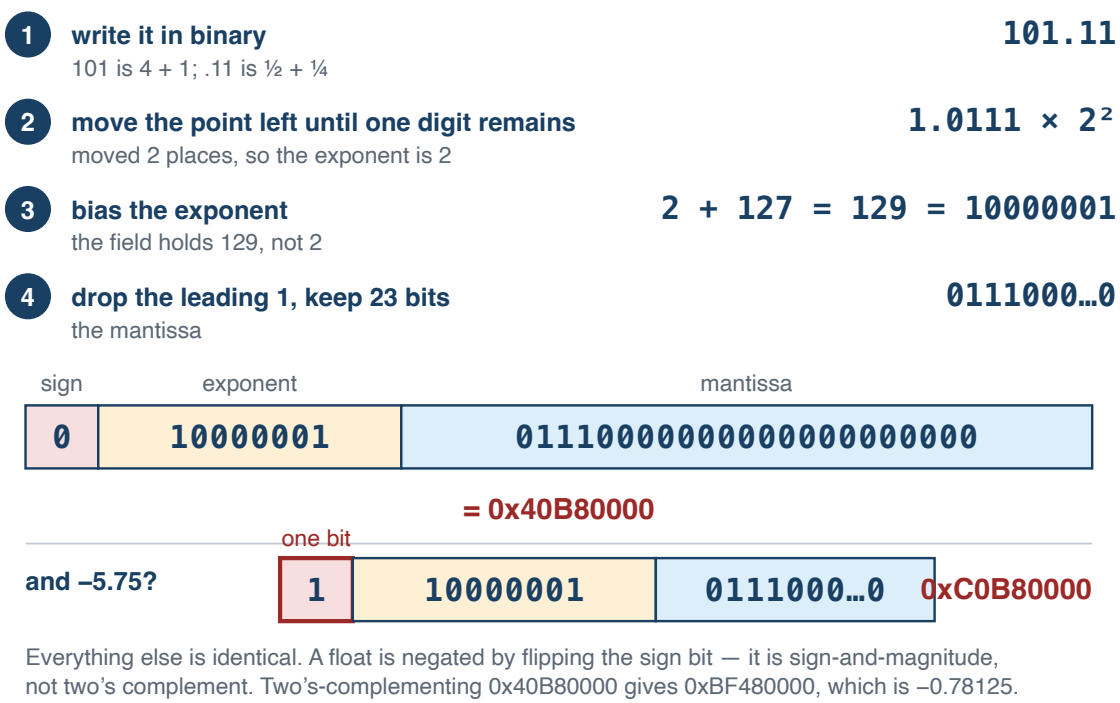
Nobody writes Avogadro's number out in full. You write a few significant digits and a power of ten. A `float` does exactly that in base 2 — and the layout is fixed by **IEEE 754**, which is why a file written on one machine reads back the same on another.

A Float Is Three Fields



A `float` is 32 bits, a `double` 64. Both spend one bit on the sign; the rest of the budget splits between **range** (exponent) and **precision** (mantissa). Every format today is a different way to split it.

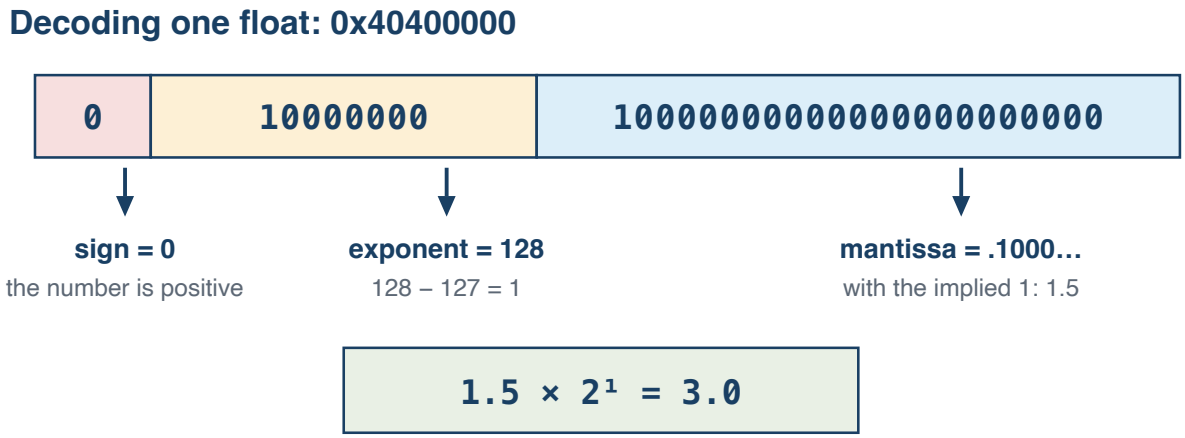
Turning 5.75 Into Thirty-Two Bits



Step 3 is the one to remember: the exponent field holds `exponent + 127`, never the exponent itself. That offset is the **bias**; for a `double` it is 1023.

And the negative? **One bit.** Floats are sign-and-magnitude, *not* two's complement — everything you learned on Monday about `~x + 1` does not apply here.

Decoding One Float



The same four steps, run backwards. Subtract the bias instead of adding it, and put the implied `1.` back in front of the mantissa.

See the Bits Yourself

A `float` and a `uint32_t` are the same 32 bits. `memcpy` moves between them, and `print_binary` from Lecture 4 prints the result:

```c
float f = 3.0f;
uint32_t bits;
memcpy(&bits, &f, sizeof bits);

printf("Hex:     %08X\n", bits);
printf("Binary:  ");
print_binary(bits);
```

```
float f = 3.0f;                  float f = 0.1f;
  Hex:     40400000                Hex:     3DCCCCCD
  Stored:  3.00000000000000000     Stored:  0.10000000149011611938
```

> **Warning**
>
> Do not reach for `*(uint32_t *)&f`. Reading an object through a pointer of an unrelated type breaks C's aliasing rules. `memcpy` is correct, and at `-O2` it compiles to the same single instruction.

Pulling the Three Fields Apart

Once you have the bits, the fields are three shifts and two masks:

```c
uint32_t sign = u >> 31;             // 1 bit
uint32_t exp  = (u >> 23) & 0xFF;    // 8 bits, biased by 127
uint32_t man  = u & 0x7FFFFFu;       // 23 bits, leading 1 not stored
```

`0x7FFFFF` is 23 one-bits — the same mask idea from Lecture 2, at a bigger scale.

Notice where the groups of four fall in `0100 0000 0100 0000 …`. Nibbles line up with **hex digits**, not with 1 / 8 / 23: the exponent starts one bit into the first group and ends on the first bit of the third. That is why `01-bits.c` groups by field instead — and why the same 32 bits are worth printing twice.

Run It: Eight Values, Thirty-Two Bits

```
    0  0x00000000  0 00000000 00000000000000000000000  +zero
   -0  0x80000000  1 00000000 00000000000000000000000  -zero
    1  0x3F800000  0 01111111 00000000000000000000000  +1          x 2^0
   -1  0xBF800000  1 01111111 00000000000000000000000  -1          x 2^0
  0.5  0x3F000000  0 01111110 00000000000000000000000  +1          x 2^-1
    3  0x40400000  0 10000000 10000000000000000000000  +1.5        x 2^1
   -3  0xC0400000  1 10000000 10000000000000000000000  -1.5        x 2^1
 5.75  0x40B80000  0 10000001 01110000000000000000000  +1.4375     x 2^2
-5.75  0xC0B80000  1 10000001 01110000000000000000000  -1.4375     x 2^2
  0.1  0x3DCCCCCD  0 01111011 10011001100110011001101  +1.60000002 x 2^-4
```

Read the pairs. Each one differs from its partner in **exactly one bit** — and that includes zero, which is why `-0.0` exists at all. `0.1` is the odd one out; look at the mantissa.

Why 0.1 Is Not 0.1



The same thing that happens to `1/3` in decimal. Nothing is broken: base 2 cannot write one tenth in finitely many digits, any more than base 10 can write one third.

Run It: What 0.1 Actually Holds

```
0.1f as float    0.1000000149011611938   0x3DCCCCCD
0.1  as double   0.1000000000000000555
0.2  as double   0.2000000000000001110
0.3  as double   0.2999999999999998890
0.1 + 0.2        0.3000000000000004441
```

Mind the type. A `float` misses 0.1 by about `1.5e-9`; a `double` by `5.6e-18`. More bits move the error — they never remove it, because one tenth has no finite binary expansion at any width.

The addition is not what went wrong. `0.1` and `0.2` were already the wrong numbers before it started.

Never Compare Floats with ==

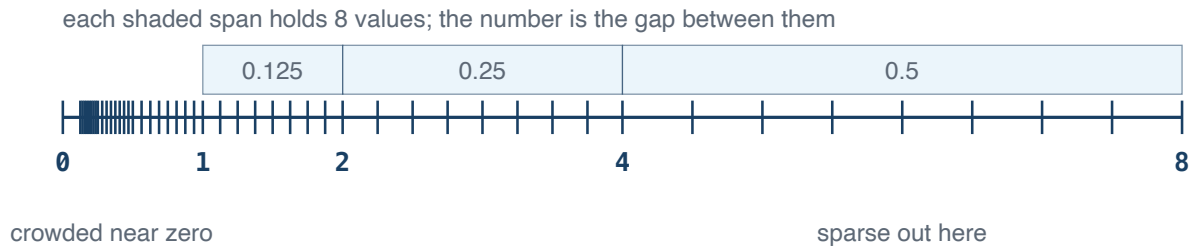
```
0.1 + 0.2 == 0.3                 // false
fabs((0.1 + 0.2) - 0.3) < 1e-9   // true — compare within a tolerance
```

Ten steps of `0.1` do not land on `1.0` either, so `for (x = 0; x != 1.0; x += 0.1)` runs forever:

```
0.1 added ten times = 0.99999999999999988898
that == 1.0 -> false
```

Count the steps with an `int`, and match the tolerance to the size of the numbers — `1e-9` is meaningless next to `1e12`.

Floats Are Not Evenly Spaced



The exponent picks a doubling; the mantissa picks a position inside it. So every doubling gets the same number of values, spread over twice the distance.

Run It: The Gap Grows With the Number

```
nextafterf(x, INFINITY) - x     // the distance to the very next float
```

```
start   next float up   gap
      1      1.00000012   1.19209e-07
      4      4.00000048   4.76837e-07
     16      16.0000019   1.90735e-06
     64      64.0000076   7.62939e-06
    256      256.000031   3.05176e-05
   1024      1024.00012   0.00012207
```

Four times the value, four times the gap. `FLT_EPSILON` is that first number, `1.19209e-07` — the gap just above 1.0, and the honest limit on a `float`'s precision.

Where a Float Stops Counting

Keep going and the gap reaches 1, then 2:

```
2^23 = 8388608   gap 1
2^24 = 16777216  gap 2

f == 16777216, f + 1 == 16777216  ->  the same number
```

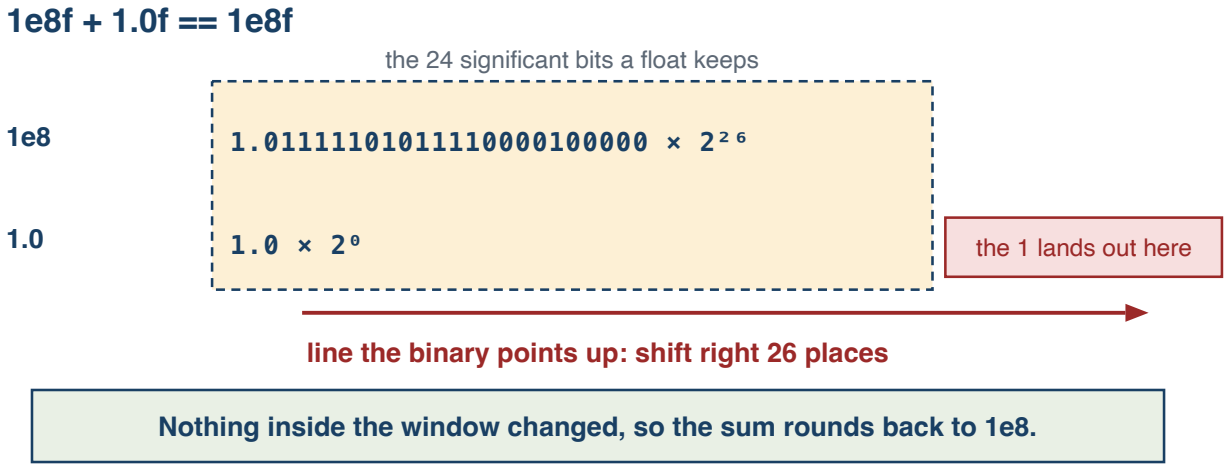
A `float` holds 24 significant bits, so past 2^24 it can no longer represent every integer. A `double` has 53, which is why an `int` survives a trip through `double` but not through `float`.

Warning

A counter kept in a `float` silently stops counting at 16,777,216. This is a real bug, and it does not

announce itself.

Big Numbers Eat Small Ones



Adding two floats means lining up their binary points first. If the exponents are far enough apart, the smaller number shifts clean off the end of the window and the sum rounds straight back to the larger one.

Floating Point Is Not Real Arithmetic

Every operation rounds its result. Change *which* results get rounded and you change the answer:

```
(0.1 + 0.2) + 0.3   // 0.60000000000000008882
0.1 + (0.2 + 0.3)   // 0.59999999999999997780
```

Addition is **not associative**. This is not a curiosity — it is why moving a sum onto more threads, or a different GPU count, or a newer library, changes the last digits of the answer.

Run It: Same Numbers, Different Order

One large value and a million `1.0f`s, summed both ways round:

```
for (int i = 0; i < N; i++)      fwd += a[i];   // big first
for (int i = N - 1; i >= 0; i--) rev += a[i];   // small first
```

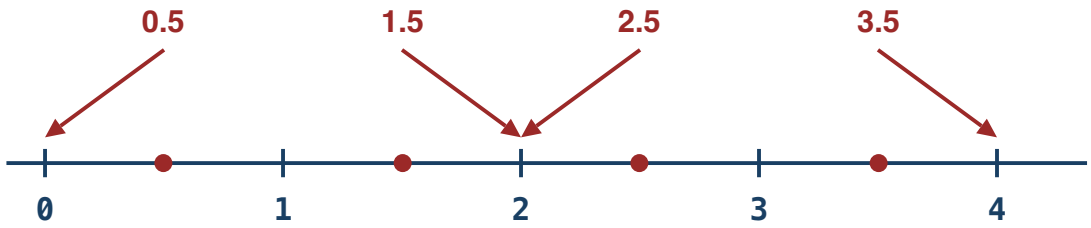
```
forwards  100000000.0
backwards 101000000.0
exact     101000000.0
```

Summing the big value first makes every `1.0f` vanish into it, one at a time — a **1% error** from nothing but the loop direction.

Rounding

Most results do not land on a representable value, so the hardware rounds. The default is **round to nearest**, and ties go to the **even** neighbour:

An exact tie rounds to the even neighbour



1.5 and 3.5 go up; 0.5 and 2.5 go down. Each lands on the even side.

Always rounding ties up would drift a long sum upward. Alternating cancels the error out.

Rounding ties upward every time would push a long sum steadily too high. Alternating cancels the error instead.

Three Ways to Lose the Fraction

They give three different answers, and only one of them is rounding:

```
printf("%.0f")  0 2 2 4 4      <- round to nearest, ties to even
round()         1 2 3 4 5      <- ties away from zero
(int) 3.9  = 3      (int) -3.9 = -3   <- truncation, not rounding
```

A cast is not a rounding function. If you want a rounded integer, say so: `(int)round(x)`.

> **Warning**
>
> Converting a floating value to an integer type is **undefined** when the value will not fit — `(int)1e10` is not "some large int", it is undefined behavior. Check the range first.

Special Values

The exponent field is reserved at both ends:

Exponent	Mantissa	Meaning
all 0s	0	±0
all 0s	nonzero	**denormalized** (very small numbers, gradual underflow)
all 1s	0	±∞
all 1s	nonzero	**NaN**

```
0.0 / 0.0    // NaN
1.0 / 0.0    // +inf
NaN == NaN   // false — always. This is why isnan() exists.
```

Run It: The Ends of the Range

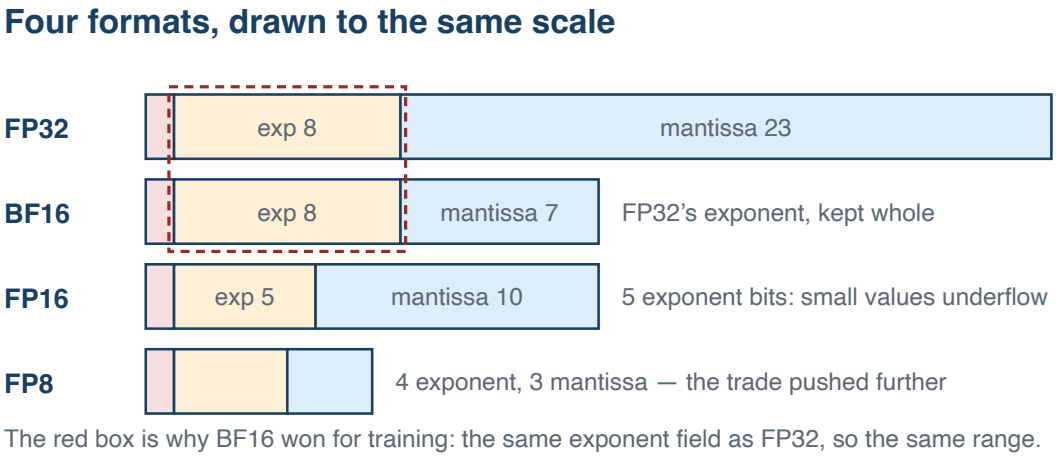
```
1.0f/0.0f        inf  0x7F800000
0.0f/0.0f        nan  0x7FC00000
-0.0f      -0.000000  0x80000000  <- a different bit pattern

0.0f == -0.0f   true    <- but they compare equal
nan == nan      false   <- always. Use isnan().
```

```
FLT_MIN       1.17549e-38  0x00800000  smallest normal
FLT_MIN/2     5.87747e-39  0x00400000  denormal: exponent field is 0
FLT_TRUE_MIN  1.4013e-45   0x00000001  one single mantissa bit
```

Below `FLT_MIN` the implied leading 1 is dropped. That buys a few more powers of ten near zero, one mantissa bit at a time, and is called **gradual underflow**.

AI Systems Connection



Every format above is the same 32-bit budget question, answered differently. Halving the bits halves the memory and roughly doubles the arithmetic throughput — the only decision is which half to give up.

BF16 Is FP32 With the Tail Cut Off

Same sign bit, same eight exponent bits, 16 fewer mantissa bits. The conversion is a truncation, which is why the hardware does it for free:

```c
static float to_bf16(float f) {
    uint32_t u = bits_of(f);
    uint32_t round = 0x7FFF + ((u >> 16) & 1);   // round to nearest even
    return from_bits((u + round) & 0xFFFF0000u); // keep the top 16 bits
}
```

```
          FP32            BF16  rel err   kept bits
     3.1415927        3.140625  -0.00031  0x40490000
           0.1      0.10009766   0.00098  0x3DCD0000
 9.9999999e-09   1.0011718e-08   0.0012   0x322C0000
```

Three decimal digits of precision, and FP32's entire range.

Two Consequences You Will Hear About

- **Loss scaling.** FP16 has only 5 exponent bits, so gradients below about `6e-8` underflow to zero. Training multiplies the loss by a large constant before the backward pass and divides it out after, moving the numbers into the range FP16 can represent. BF16 keeps FP32's exponent, so it needs none of this — which is why it won.
- **Irreproducible training runs.** Because FP addition is not associative, summing gradients in a different order on a different GPU count gives a different result. Same code, same data, different answer.

The Unit in One Slide

Every value you meet from here is one of four shapes, and you can now read all of them:

	what it is	where the surprise was
`int`, `unsigned`	bits in a fixed-width box	the box is too small
`int *`	a number that names a byte	what it names can be gone
`struct`	fields side by side, with gaps	the gaps
`float`	sign, exponent, mantissa	it is not the number you typed

Nothing in memory says which. **The type is the only thing that decides how the bits are read** — which is the one sentence the whole unit has been making.

Next: **Assembly**. The same bytes, from the instruction side — and the first thing you will do is recognise these four in a register dump.

Practice

1. Decode `0xC0A00000` as a `float` by hand. What decimal value is it?
2. Build the float for `-2.5` by hand, the way we built `5.75`. Check yourself with `01-bits.c`.
3. Sum an array of floats forwards, then backwards. Do you get the same answer? Which one is closer to the truth, and why?
4. `0.1 + 0.2 == 0.3` is false. Print all three to 20 decimal places and explain what you see.
5. Why does a `float` counter stop advancing at 16,777,216? At what value does a `double` counter stop?
6. Why does BF16 keep 8 exponent bits when FP16 only has 5?

Reading

- CS:APP §2.4 (Floating Point)
- CS:APP §2.5 (Summary of Integer and Floating-Point Arithmetic)